

ATIVIDADE – DERIVAÇÃO DE UM CÓDIGO A PARTIR DA GRAMÁTICA DA LINGUAGEM C

Linguagem escolhida: C

Bruno Vinicius Coutinho Miente - RA: 24151605-2

João Marcos Moreira Nunes - RA: 24055168-2

Luiz Otavio Roder - RA: 24159236-2

Matheus Torrezan Izepe - RA: 24170436-2

Vitoria Souza Santos - RA: 24229424-2

Turma: ESOFT5S-C

1. Linguagem escolhida

A linguagem escolhida para esta atividade é a linguagem C. C é uma linguagem de programação compilada, estruturada e de propósito geral. Sua sintaxe pode ser descrita formalmente por meio de regras gramaticais que definem como tokens e construções da linguagem podem ser organizados.

Para esta atividade, será considerada uma pequena parte da gramática necessária para representar uma declaração de variável com uma expressão aritmética.

2. Fonte da gramática consultada

A gramática utilizada como referência é baseada na especificação da linguagem C definida pelo padrão ISO/IEC 9899, conhecido como padrão da linguagem C.

Fonte online consultada:

C Grammar — [cppreference.com](https://en.cppreference.com)

<https://en.cppreference.com/w/c/language.html>

A gramática de C é tradicionalmente apresentada utilizando uma notação semelhante à BNF (Backus-Naur Form), com algumas extensões e convenções próprias da especificação.

3. Código escolhido

O trecho de código que será derivado é:

```
int resultado = 10 + 20;
```

Esse código realiza uma declaração de variável chamada resultado, do tipo int, inicializada com o resultado da expressão 10 + 20.

4. Produções selecionadas

Para simplificar a atividade, serão utilizadas as seguintes regras, baseadas na estrutura gramatical da linguagem C:

```
<declaracao> ::= <declaracao-de-variavel> ";"  
<declaracao-de-variavel> ::= <tipo> <identificador> "=" <expressao>  
<tipo> ::= "int"  
<identificador> ::= <nome>  
<nome> ::= "resultado"  
<expressao> ::= <numero> "+" <numero>  
<numero> ::= "10" | "20"
```

Significado das principais regras:

- **<declaracao>**: Representa uma declaração completa em C. A declaração termina com ponto e vírgula.
- **<declaracao-de-variavel>**: Representa a criação de uma variável, contendo seu tipo, nome e valor inicial.
- **<tipo>**: Representa o tipo da variável. Neste exemplo, será utilizado int.
- **<identificador>**: Representa o nome utilizado para identificar a variável.
- **<expressao>**: Representa uma expressão matemática composta por dois números e pelo operador +.
- **<numero>**: Representa um valor numérico. Para este exemplo, serão utilizados 10 e 20.

5. Derivação passo a passo

O símbolo inicial da gramática será:

```
<declaracao>
```

Queremos chegar ao código:

```
int resultado = 10 + 20;
```

Passo 1 — Símbolo inicial

Expressão atual:

```
<declaracao>
```

Regra aplicada:

```
<declaracao> ::= <declaracao-de-variavel> ";"
```

Resultado:

```
<declaracao-de-variavel> ";"
```

Passo 2 — Declaração da variável

Expressão atual:

```
<declaracao-de-variavel> ";"
```

Regra aplicada:

<declaracao-de-variavel> ::= <tipo> <identificador> "=" <expressao>

Resultado:

<tipo> <identificador> "=" <expressao> ";;"

Passo 3 — Definição do tipo

Expressão atual:

<tipo> <identificador> "=" <expressao> ";;"

Regra aplicada:

<tipo> ::= "int"

Resultado:

"int" <identificador> "=" <expressao> ";;"

Passo 4 — Definição do identificador

Expressão atual:

"int" <identificador> "=" <expressao> ";;"

Regra aplicada:

<identificador> ::= <nome>

Resultado:

"int" <nome> "=" <expressao> ";;"

Passo 5 — Definição do nome da variável

Expressão atual:

"int" <nome> "=" <expressao> ";;"

Regra aplicada:

<nome> ::= "resultado"

Resultado:

"int" "resultado" "=" <expressao> ";;"

Passo 6 — Definição da expressão

Expressão atual:

"int" "resultado" "=" <expressao> ";;"

Regra aplicada:

<expressao> ::= <numero> "+" <numero>

Resultado:

"int" "resultado" "=" <numero> "+" <numero> ";;"

Passo 7 — Primeiro número

Expressão atual:

```
"int" "resultado" "=" <numero> "+" <numero> ";;"
```

Regra aplicada:

```
<numero> ::= "10"
```

Resultado:

```
"int" "resultado" "=" "10" "+" <numero> ";;"
```

Passo 8 — Segundo número

Expressão atual:

```
"int" "resultado" "=" "10" "+" <numero> ";;"
```

Regra aplicada:

```
<numero> ::= "20"
```

Resultado:

```
"int" "resultado" "=" "10" "+" "20" ";;"
```

Removendo as aspas utilizadas apenas para representar os símbolos terminais, obtemos o código:

```
int resultado = 10 + 20;
```

6. Derivação completa resumida

```
<declaracao>  
⇒ <declaracao-de-variavel> ";;"  
⇒ <tipo> <identificador> "=" <expressao> ";;"  
⇒ "int" <identificador> "=" <expressao> ";;"  
⇒ "int" <nome> "=" <expressao> ";;"  
⇒ "int" "resultado" "=" <expressao> ";;"  
⇒ "int" "resultado" "=" <numero> "+" <numero> ";;"  
⇒ "int" "resultado" "=" "10" "+" <numero> ";;"  
⇒ "int" "resultado" "=" "10" "+" "20" ";;"
```

Resultado final:

```
int resultado = 10 + 20;
```

7. Símbolos terminais e não terminais

Na gramática utilizada, existem símbolos não terminais e símbolos terminais.

Símbolos não terminais:

```
<declaracao>  
<declaracao-de-variavel>
```

<tipo>
<identificador>
<nome>
<expressao>
<numero>

Os símbolos não terminais ainda podem ser substituídos por outras regras da gramática. Eles aparecem entre < e >.

Símbolos terminais:

```
int
resultado
=
10
+
20
;
```

Os símbolos terminais são aqueles que aparecem efetivamente no código final e não precisam mais ser substituídos.

8. Relação entre a gramática e o código

A derivação demonstra que o código não é formado de maneira aleatória. Cada parte possui uma função definida pela gramática.

```
int      → tipo
resultado → identificador
=        → operador de atribuição
10       → número
+        → operador aritmético
20       → número
;        → final da declaração
```

A gramática determina como esses elementos podem ser organizados. A partir do símbolo inicial <declaracao>, foram aplicadas sucessivamente as produções até que todos os símbolos não terminais fossem substituídos por símbolos terminais.

9. Resultado final

O resultado da derivação foi:

```
int resultado = 10 + 20;
```

Esse é um trecho válido da linguagem C. A derivação começou com o símbolo não terminal <declaracao> e, por meio das regras de produção selecionadas, chegou à sequência de símbolos terminais que compõem o código.

Dessa forma, é possível observar na prática a relação entre gramáticas formais, símbolos terminais, símbolos não terminais, produções e derivação dentro de uma linguagem de programação real.